CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 5

NAME: Kanishk Jamwal	**UID:** 24BAI70827
BRANCH: CSE-AIML (CS221)	**SECTION/GROUP:** 24AIT-NTPP-3[A]
SEMESTER: 5TH	**DATE OF PERFORMANCE:** 19/08/2026
SUBJECT: Full Stack - II	**SUBJECT CODE:** 24CSP-337

EXPERIMENT 5.1

1. AIM

To design and implement RESTful APIs using Spring Boot with proper validation, standardized responses, and scalable architecture.

2. OBJECTIVE

1. To understand REST API design principles.
2. To implement CRUD APIs using Spring Boot.
3. To enforce consistent request-response structures.
4. To apply validation using Bean Validation.
5. To enable secure cross-origin communication.

3. SOFTWARE REQUIREMENT

1.Java (JDK 17+)
2.Spring Boot
3.IntelliJ IDEA / Eclipse / VS Code
4. Postman / API testing tool
5. Maven
6. Web Browser

4. THEORY

REST (Representational State Transfer) is an architectural style used for designing scalable and loosely coupled web services. In RESTful systems, data is treated as resources and accessed through standard HTTP methods such as GET, POST, PUT, and DELETE.

Spring Boot simplifies REST API development by providing a layered architecture consisting of:

Controller Layer → Handles HTTP requests.

Service Layer → Contains business logic.

Repository Layer → Interacts with the database.

A well-designed REST API should follow stateless communication, consistent URI structures, and standardized response formats.

To ensure data integrity, Bean Validation is applied using annotations such as @NotNull, @NotBlank, and @Size. These annotations validate incoming request data before it is processed by the application.

Cross-Origin Resource Sharing (CORS) allows secure communication between frontend and backend applications running on different origins. Proper CORS configuration enables the frontend to access backend APIs safely.

Standardized API responses provide a predictable structure for both successful and failed requests, making frontend-backend integration easier and more maintainable.

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot project using Spring Initializr.
2. Add required dependencies such as Spring Web, Spring Data JPA, Validation, and database support.
3. Create the **Post entity** for storing post information.
4. Create the **Repository layer** to interact with the database.
5. Create the **Service layer** to implement business logic.
6. Create the **Controller layer** to expose REST endpoints.
7. Implement CRUD operations using GET, POST, PUT, and DELETE.
8. Apply Bean Validation annotations to request fields.
9. Create a standardized API response structure.
10. Configure CORS to allow frontend-backend communication.
11. Run the Spring Boot application.
12. Test the REST APIs using Postman.
13. Verify successful and invalid requests.

6. CODE

Post.java

```
package com.example.demo.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;

@Entity
public class Post {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @NotBlank(message = "Title is required")
   @Size(min = 3, max = 100)
   private String title;

   @NotBlank(message = "Content is required")
   private String content;

   public Long getId() {
      return id;
   }

   public void setId(Long id) {
      this.id = id;
   }

   public String getTitle() {
      return title;
   }

   public void setTitle(String title) {
      this.title = title;
   }
```

```
    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }
}
```

PostRepository.java

```
package com.example.demo.repository;

import com.example.demo.model.Post;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface PostRepository extends JpaRepository<Post, Long> {
}
```

PostService.java

```
package com.example.demo.service;

import com.example.demo.model.Post;
import com.example.demo.repository.PostRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class PostService {

    private final PostRepository repository;

    public PostService(PostRepository repository) {
        this.repository = repository;
    }

    public List<Post> getAllPosts() {
        return repository.findAll();
    }

    public Post getPost(Long id) {
        return repository.findById(id).orElse(null);
    }

    public Post createPost(Post post) {
        return repository.save(post);
    }
```

```
    public Post updatePost(Long id, Post post) {
        Post existing = repository.findById(id).orElse(null);
        if (existing != null) {
            existing.setTitle(post.getTitle());
            existing.setContent(post.getContent());
            return repository.save(existing);
        }
        return null;
    }

    public void deletePost(Long id) {
        repository.deleteById(id);
    }
}
```

PostController.java

```
package com.example.demo.controller;
import com.example.demo.model.Post;
import com.example.demo.service.PostService;
import jakarta.validation.Valid;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/posts")
@CrossOrigin(origins = "*")
public class PostController {

    private final PostService service;

    public PostController(PostService service) {
        this.service = service;
    }

    @GetMapping
    public List<Post> getAllPosts() {
        return service.getAllPosts();
    }

    @GetMapping("/{id}")
    public Post getPost(@PathVariable Long id) {
        return service.getPost(id);
    }

    @PostMapping
    public Post createPost(@Valid @RequestBody Post post) {
        return service.createPost(post);
    }

    @PutMapping("/{id}")
    public Post updatePost(@PathVariable Long id, @Valid @RequestBody Post post) {
```

```
        return service.updatePost(id, post);
    }

    @DeleteMapping("/{id}")
    public String deletePost(@PathVariable Long id) {
        service.deletePost(id);
        return "Post deleted successfully";
    }
}
```

Standard API Response

```
package com.example.demo.dto; // Or .payload / .response depending on your packaging strategy

public class ApiResponse<T> {

    private boolean success;
    private String message;
    private T data;

    public ApiResponse(boolean success, String message, T data) {
        this.success = success;
        this.message = message;
        this.data = data;
    }

    public boolean isSuccess() {
        return success;
    }

    public void setSuccess(boolean success) {
        this.success = success;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public T getData() {
        return data;
    }

    public void setData(T data) {
        this.data = data;
    }
}
```

7. OUTPUT

The RESTful API system was successfully implemented using Spring Boot. The application provided CRUD operations for managing posts through REST endpoints using GET, POST, PUT, and DELETE methods. Bean Validation was applied to ensure that incoming request data satisfied the required constraints. The application followed a layered architecture consisting of Controller, Service, and Repository layers. CORS configuration enabled communication between frontend and backend systems running on different origins. The APIs were tested using Postman and produced structured responses for successful operations and validation errors.

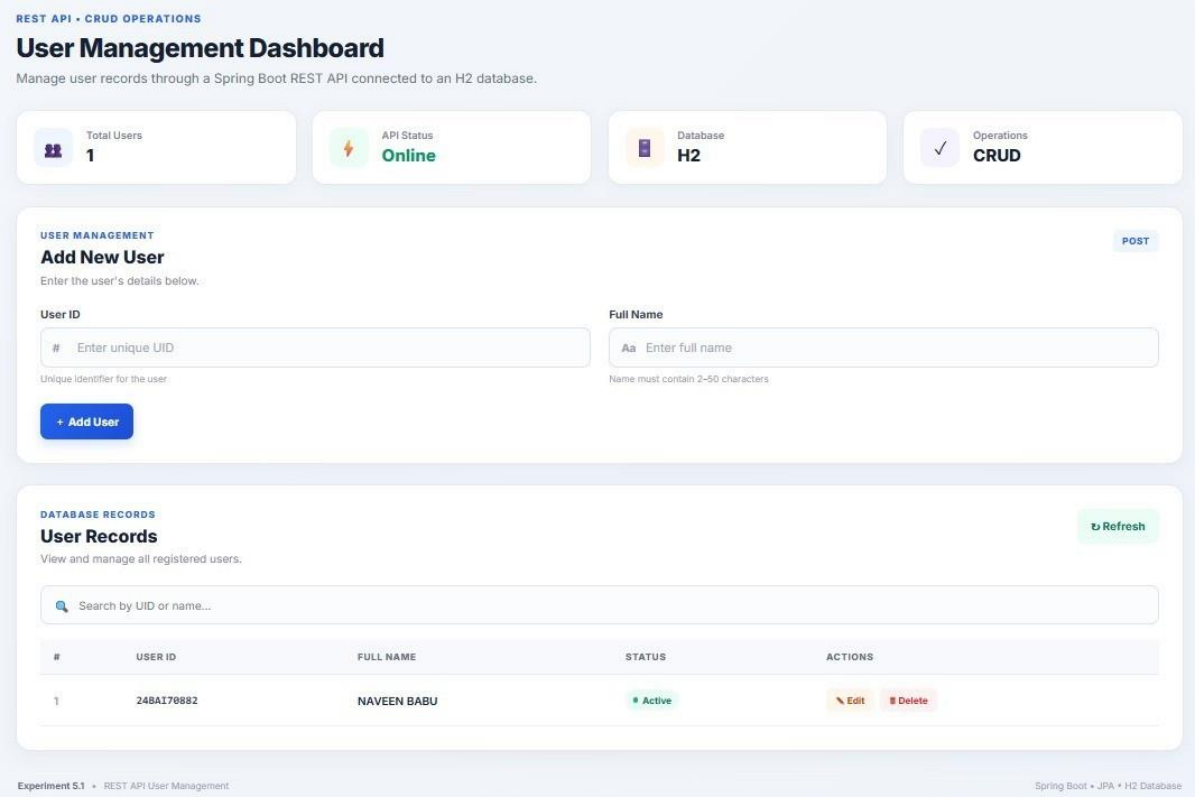


Fig: 5.1 RESTful API implementation using Spring Boot

8. LEARNING OUTCOMES

- Understand the basic principles of RESTful API design.
- Develop CRUD APIs using Spring Boot.
- Understand Controller, Service, and Repository layers.
- Implement REST APIs using HTTP methods such as GET, POST, PUT, and DELETE.
- Apply Bean Validation for validating incoming request data.
- Implement standardized API request and response structures.
- Configure CORS for frontend-backend communication.
- Test and verify REST APIs using Postman.
- Develop scalable and maintainable backend applications using Spring Boot.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT 5.2

1. AIM

To implement global exception handling and structured logging for building robust and observable backend systems using Spring Boot.

2. OBJECTIVE

1. To handle exceptions centrally using @ControllerAdvice.
2. To implement logging mechanisms for request tracking.
3. To use correlation IDs for tracing requests.
4. To improve system observability and debugging.

3. SOFTWARE REQUIREMENT

1. Java (JDK 17+)
2. Spring Boot
3. IntelliJ IDEA / Eclipse / VS Code
4. Postman / API testing tool
5. Logging framework (SLF4J / Logback)
6. Maven
7. Web Browser

4. THEORY

In large-scale applications, handling exceptions individually in each controller can lead to code duplication and inconsistent error responses. Spring Boot provides global exception handling using @ControllerAdvice, which centralizes exception handling and ensures uniform error responses across the application.
Global exception handling improves: Code maintainability,Error consistency and Debugging efficiency
Logging is also an important part of backend development because it helps monitor application behavior and identify errors. Logging mechanisms can capture information such as request paths, HTTP methods, execution time, and exception details.
Filters and Interceptors act as middleware components that intercept incoming requests and outgoing responses. They can be used for: Logging,Authentication,Request modification and Request tracking
To improve traceability, a correlation ID can be assigned to every request. Using MDC (Mapped Diagnostic Context), this ID can be included in application logs. This allows developers to trace the complete lifecycle of a request across different log messages.Together, global exception handling, structured logging, filters, interceptors, and correlation IDs help create a robust, observable, and production-ready backend system.

5. IMPLEMENTATION/PROCEDURE

1.Create a Spring Boot project using Spring Initializr.
2.Add required Spring Web and validation dependencies.
3.Create the **Post entity** and REST controller.
4.Create a custom exception class for handling missing resources.
5.Implement global exception handling using @ControllerAdvice.
6.Create a standardized error response structure.
7.Configure SLF4J/Logback for application logging.
8. Create a logging filter to intercept incoming requests.
9. Generate a unique correlation ID for every request.
10. Store the correlation ID using MDC.
11. Log request method, URI, correlation ID, and execution time.
12. Test successful and erroneous requests using Postman.
13. Verify that exceptions are handled centrally and logs contain correlation IDs.

6. **CODE**

PostNotFoundException.java

```
package com.example.demo.exception;

public class PostNotFoundException extends RuntimeException {

    public PostNotFoundException(Long id) {
        super("Post not found with id: " + id);
    }
}
```

ErrorResponse.java

```
package com.example.demo.exception;

public class ErrorResponse {

    private boolean success;
    private String message;
    private String correlationId;

    public ErrorResponse(boolean success, String message, String correlationId) {
        this.success = success;
        this.message = message;
        this.correlationId = correlationId;
    }

    public boolean isSuccess() {
        return success;
    }

    public String getMessage() {
        return message;
    }

    public String getCorrelationId() {
        return correlationId;
    }
}
```

PostService.java

```
package com.example.demo.service;

import com.example.demo.exception.PostNotFoundException;
import com.example.demo.model.Post;
import com.example.demo.repository.PostRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class PostService {
```

```
    private final PostRepository repository;

    public PostService(PostRepository repository) {
        this.repository = repository;
    }

    public List<Post> getAllPosts() {
        return repository.findAll();
    }

    public Post getPost(Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new PostNotFoundException(id));
    }

    public Post createPost(Post post) {
        return repository.save(post);
    }
}
```

GlobalExceptionHandler.java

```
package com.example.demo.exception;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(PostNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handlePostNotFound(PostNotFoundException ex) {
        String correlationId = MDC.get("correlationId");
        logger.error("Post not found: {}", ex.getMessage());
        return new ErrorResponse(false, ex.getMessage(), correlationId);
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleException(Exception ex) {
        String correlationId = MDC.get("correlationId");
        logger.error("Unexpected error: {}", ex.getMessage());
        return new ErrorResponse(false, "Internal server error", correlationId);
    }
```

```
}
```

RequestLoggingFilter.java

```
package com.example.demo.filter;

import jakarta.servlet.*;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.util.UUID;

@Component
public class RequestLoggingFilter implements Filter {

    private static final Logger logger = LoggerFactory.getLogger(RequestLoggingFilter.class);

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        String correlationId = UUID.randomUUID().toString();
        MDC.put("correlationId", correlationId);

        long start = System.currentTimeMillis();
        logger.info("Request started: {} {}", req.getMethod(), req.getRequestURI());

        try {
            chain.doFilter(request, response);
        } finally {
            long time = System.currentTimeMillis() - start;
            logger.info("Request completed in {} ms", time);
            MDC.clear();
        }
    }
}
```

PostController.java

```
package com.example.demo.controller;

import com.example.demo.model.Post;
import com.example.demo.service.PostService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
```

```
@RequestMapping("/api/posts")
public class PostController {

    private final PostService service;

    public PostController(PostService service) {
        this.service = service;
    }

    @GetMapping
    public List<Post> getAllPosts() {
        return service.getAllPosts();
    }

    @GetMapping("/{id}")
    public Post getPost(@PathVariable Long id) {
        return service.getPost(id);
    }

    @PostMapping
    public Post createPost(@RequestBody Post post) {
        return service.createPost(post);
    }
}
```

7. OUTPUT

The global exception handling and structured logging system was successfully implemented using Spring Boot. The application centrally handled exceptions using @RestControllerAdvice and returned consistent error responses.A request logging filter was implemented to track incoming requests and execution time. A unique correlation ID was generated for every request and stored using MDC. The correlation ID helped trace requests across different log messages.The system was tested using Postman for both successful and invalid requests. Errors were handled centrally without duplicating exception-handling code in individual controllers.

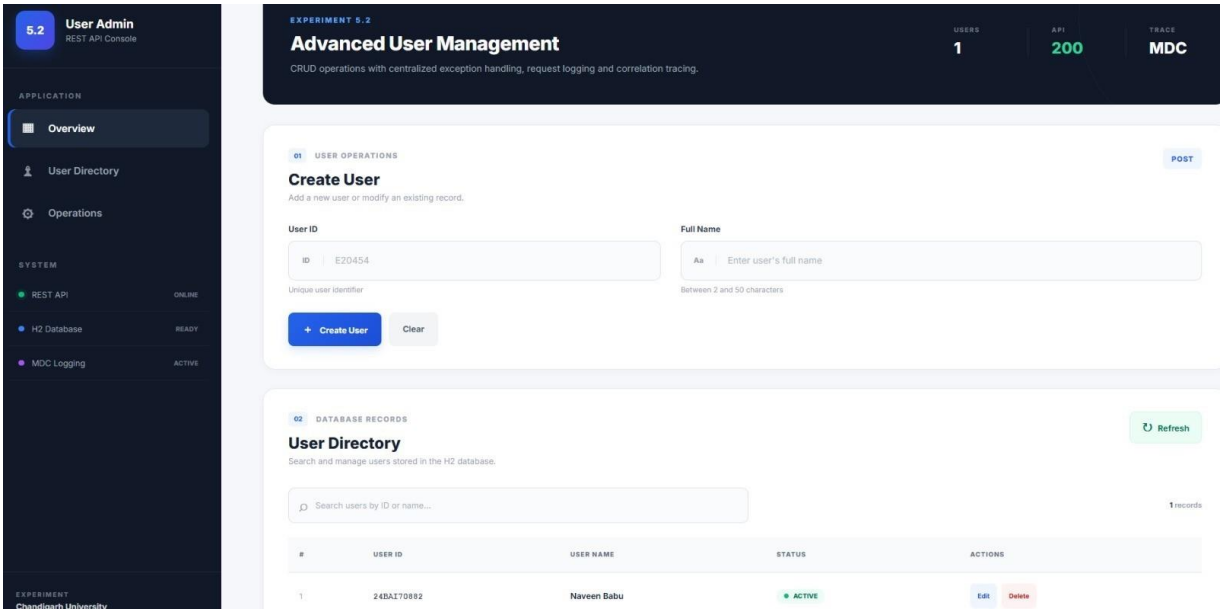


Fig: 5.2 Global exception handling and structured logging using Spring Boot

8. LEARNING OUTCOMES

- Understand the importance of global exception handling.
- Implement centralized exception handling using @ControllerAdvice.
- Create and use custom exception classes.
- Implement structured logging using SLF4J and Logback.
- Understand the role of Filters and Interceptors in Spring Boot.
- Generate and use correlation IDs for request tracing.
- Use MDC for maintaining request-specific logging information.
- Improve backend debugging and monitoring.
- Create consistent error responses for REST APIs.
- Develop more robust and observable Spring Boot applications.